BUG BOUNTY HUNTING

A lab guide in five hands-on exercises — from building a safe laboratory to writing a professional vulnerability report.

COURSE LEVEL	TOTAL TIME	LABS	COST
Beginner	**10–15 hours**	**5 + assessment**	**Free tools only**

Prerequisites

Basic computer skills, a modern web browser, and familiarity with typing commands into a terminal. No prior security experience is required. A computer with at least 8 GB RAM and 10 GB free disk space is recommended.

Contents

Part A	Principles of Bug Bounty Hunting
Part B	Software and Laboratory Configuration Guide
Lab 01	Reconnaissance and Attack Surface Mapping
Lab 02	Understanding HTTP and Intercepting Requests with Burp Suite
Lab 03	SQL Injection Fundamentals
Lab 04	Cross-Site Scripting (XSS)
Lab 05	Broken Access Control and Writing a Bug Bounty Report
Final	Practical Assessment and Grading Rubric
Appendix	Instructor Guidance, Troubleshooting, Further Learning

How to read each lab

Every lab follows the same structure: **objectives**, **background**, numbered **exercises** with exact commands, and a framed **Expected result** panel that tells you what you should see — and *why* you see it. Each lab closes with an assignment, a knowledge check and a summary of what you learned.

PART A

Principles of Bug Bounty Hunting

What is bug bounty hunting?

Bug bounty hunting is the process of identifying and responsibly reporting security vulnerabilities in systems that an organization has **explicitly authorized** researchers to test. Organizations run bug bounty programs to find weaknesses before malicious attackers discover and exploit them, and they reward researchers — with money, recognition or both — for valid, well-documented findings.

A successful hunter must understand how applications work, how vulnerabilities occur, how to reproduce findings safely, and how to communicate risk clearly to developers and security teams.

How a bug bounty program works

ELEMENT	WHAT IT MEANS FOR YOU
Scope	The exact domains, applications and APIs you may test. Anything not listed is out of scope and must not be touched.
Rules of engagement	Allowed techniques, traffic limits, account rules and data-handling requirements. Automated scanning is often restricted.
Safe harbor	The organization's commitment not to pursue legal action against good-faith research that follows the rules.
Triage	The program team reproduces your report, confirms validity and assigns severity.
Reward	Paid according to severity and impact. Duplicates and out-of-scope findings usually receive nothing.
Disclosure	Findings stay confidential until the organization fixes them and approves any public disclosure.

The researcher's workflow

Every lab in this guide practises part of one repeatable method:

01	02	03	04	05	06
Discover	**Observe**	**Test**	**Validate**	**Remediate**	**Report**
Map what exists	Watch normal behaviour	Change one thing	Prove it is real	Know the fix	Communicate clearly

Five rules of ethical hunting

1. **Obtain authorization.** Only test applications explicitly included in a program's scope.
2. **Follow program rules.** Respect restrictions on testing methods, accounts, traffic volume and data handling.
3. **Minimize impact.** Avoid disrupting services or accessing information beyond what is necessary to demonstrate a finding.
4. **Protect information.** Do not collect, retain or disclose real user information unnecessarily.
5. **Report responsibly.** Submit clear, reproducible findings through the organization's designated disclosure channel.

LABORATORY SCOPE FOR THIS COURSE

All testing in this guide is performed only against applications running on your own computer: OWASP Juice Shop, OWASP WebGoat, and two small files you create yourself. Completing these labs **does not** authorize testing against any public website, employer system or third-party application.

Vulnerability classes covered

LAB	SKILL OR VULNERABILITY CLASS	OWASP TOP 10 CATEGORY
01	Reconnaissance and attack surface mapping	Foundation for all testing
02	HTTP analysis and request interception	Foundation for all testing
03	SQL injection	Injection
04	Cross-site scripting (XSS)	Injection
05	Insecure direct object reference (IDOR)	Broken Access Control

PART B

Software and Laboratory Configuration

You will build an isolated laboratory from intentionally vulnerable applications and free, open-source security tools. Everything runs on your own machine and is bound to the loopback address `127.0.0.1`, so nothing is reachable from your network.

Required software

SOFTWARE	PURPOSE	USED IN
Fedora, Ubuntu, Windows or macOS	Laboratory operating system	All
Docker	Runs the training applications in isolated containers	All
OWASP Juice Shop	Intentionally vulnerable online shop	01, 02
OWASP WebGoat	Guided, lesson-based security training application	03, 04
Firefox or Chrome	Browsing, Developer Tools and local HTML testing	01, 04
Burp Suite Community Edition	Intercepting proxy for HTTP inspection and replay	02
curl	Sends HTTP requests from the terminal	01, 05
Python 3	Runs the local access-control training server	05
Nmap	Discovers open ports and network services	01

Step 1 — Install Docker and the command-line tools

Choose the instructions for your operating system.

Fedora Linux

TERMINAL · FEDORA

```
sudo dnf update -y
sudo dnf install -y moby-engine docker-compose nmap curl python3 git
sudo systemctl enable --now docker
```

Ubuntu Linux

TERMINAL · UBUNTU

```
sudo apt update
sudo apt install -y docker.io docker-compose-v2 nmap curl python3 git
sudo systemctl enable --now docker
```

Windows and macOS

1. Install **Docker Desktop** from docs.docker.com/get-docker. On Windows, accept the prompt to enable WSL 2, then restart.
2. Start Docker Desktop and wait until it reports that the engine is running.
3. Install **Nmap** from nmap.org/download.html (Windows includes the Npcap driver — accept it).
4. Install **Python 3** from python.org/downloads. On Windows, tick *Add python.exe to PATH*.
5. **curl** is built into Windows 10/11 and macOS. On Windows and macOS, omit `sudo` from every Docker command in this guide; on Windows use `python` where the guide says `python3`, and write multi-line commands on one line.

Verify Docker

TERMINAL

```
sudo docker --version
sudo docker info
sudo docker run --rm hello-world
```

EXPECTED RESULT

A version number, a block of engine details, and a message beginning *"Hello from Docker!"*.

WHY

The hello-world container proves Docker can download an image from the registry, create a container and run it — exactly what the next two steps require.

Step 2 — Install OWASP Juice Shop

Juice Shop is a deliberately insecure online store containing dozens of realistic vulnerabilities. Download the image, then start it bound to loopback only:

TERMINAL

```
sudo docker pull bkimminich/juice-shop

sudo docker run -d \
  --name juice-shop \
  -p 127.0.0.1:3000:3000 \
  bkimminich/juice-shop
```

Open http://localhost:3000 in your browser.

EXPECTED RESULT

The Juice Shop storefront loads with a product grid and a welcome banner (dismiss it).

WHY

`-d` runs the container in the background, `--name` lets you start and stop it by name later, and `-p 127.0.0.1:3000:3000` publishes port 3000 to your machine only — not to your Wi-Fi network.

Step 3 — Install OWASP WebGoat

WebGoat is a guided teaching application: each lesson explains a vulnerability and then asks you to exploit it inside a controlled exercise.

TERMINAL

```
sudo docker pull webgoat/webgoat

sudo docker run -d \
  --name webgoat \
  -p 127.0.0.1:8080:8080 \
  -p 127.0.0.1:9090:9090 \
  webgoat/webgoat
```

1. Wait about 30–60 seconds for startup, then open http://localhost:8080/WebGoat.
2. Click **Register new user**. Create a training account (for example `student01`) with a password you will remember. This account exists only inside your container.
3. Log in. The lesson menu appears on the left.

EXPECTED RESULT

You are logged into WebGoat and can see lesson categories such as *(A3) Injection*. Lesson names and numbering vary between WebGoat releases.

WHY

WebGoat takes up port **8080** (the lessons) and port 9090 (WebWolf, a helper app used in some lessons). Note that 8080 is now occupied — you will configure Burp around this in Step 4.

Step 4 — Install and configure Burp Suite Community Edition

1. Visit portswigger.net/burp/communitydownload.
2. Download the Community Edition installer for your operating system and run it with the default options.
3. Start Burp Suite. Choose **Temporary project in memory** → **Next** → **Use Burp defaults** → **Start Burp**.
4. **Move Burp's proxy listener off port 8080.** Burp listens on 127.0.0.1:8080 by default, which WebGoat already uses. Open **Proxy → Proxy settings** (or **Settings → Tools → Proxy**), select the listener, click **Edit**, change the port to `8081` and confirm the listener's *Running* box is ticked.
5. Go to **Proxy → Intercept** and make sure interception is **off** for now.
6. Click **Open browser**. This launches Burp's built-in Chromium browser, already configured to send its traffic through Burp — no certificate or proxy setup is needed.
7. In that browser, visit http://localhost:3000, then open **Proxy → HTTP history**.

EXPECTED RESULT

HTTP history fills with requests to `localhost:3000` as Juice Shop loads.

WHY

Burp sits between the browser and the server as a "man in the middle" that you control. Every request passes through it, so it can record, pause and replay traffic. If history stays empty, see *Troubleshooting* in the appendix.

Step 5 — Verify the complete environment

TERMINAL

```
sudo docker ps
curl -I http://localhost:3000
curl -I http://localhost:8080/WebGoat/login
nmap --version
python3 --version
```

CHECK	WHAT SUCCESS LOOKS LIKE
`docker ps`	Two rows: *juice-shop* and *webgoat*, both with status *Up*
curl to port 3000	`HTTP/1.1 200 OK`
curl to WebGoat	`HTTP/1.1 200` (or a 302 redirect)
nmap / python3	A version number for each, with no "command not found"
Burp Suite	Built-in browser loads Juice Shop; HTTP history records it

Starting, stopping and resetting the lab

TERMINAL

```
# At the end of a session
sudo docker stop juice-shop webgoat

# At the start of the next session
sudo docker start juice-shop webgoat

# Reset Juice Shop to a clean state (removes all progress)
sudo docker rm -f juice-shop
sudo docker run -d --name juice-shop -p 127.0.0.1:3000:3000 bkimminich/juice-shop
```

IMPORTANT — KEEP THE LAB ISOLATED

Always publish ports with the `127.0.0.1:` prefix. Never expose these vulnerable applications to your home network or the public internet.

LAB 01

Reconnaissance and Attack Surface Mapping

Duration · 2 hours | Difficulty · Beginner | Target · Juice Shop | Tools · Nmap, curl, DevTools

Learning objectives

- Explain active and passive reconnaissance.
- Identify open ports and services.
- Recognize web application entry points.
- Map the visible attack surface.
- Document observations without assuming every exposed feature is vulnerable.

Background

Reconnaissance is the process of gathering information about a target before security testing. **Passive reconnaissance** collects information without directly interacting with the target — for example, reading public documentation. **Active reconnaissance** interacts with the system directly — requesting pages or scanning authorized ports. In this lab, all active testing occurs against your local Juice Shop.

Exercise 1 — Identify running services

Step 1. Confirm Juice Shop is running.

```
sudo docker ps
```

Step 2. Scan the local Juice Shop port.

```
nmap -sT -sV -p 3000 127.0.0.1
```

Step 3. Record whether TCP port 3000 is open, any service information Nmap reports, and any limitations in the service identification.

EXPECTED RESULT

Port `3000/tcp` is reported **open**. The service column may show *ppp?*, *http* or an unrecognized fingerprint rather than "Juice Shop".

WHY

`-sT` completes a full TCP handshake (no admin privileges needed); `-sV` probes the service to guess what it is. Nmap guesses from known fingerprints — a custom Node.js app has none, so the label is uncertain. A good researcher records it as a guess, not a fact.

Exercise 2 — Examine HTTP responses

```
curl -i http://localhost:3000/
```

Identify: (1) the HTTP status code, (2) server-related headers, if present, (3) the Content-Type, and (4) any visible security headers such as `X-Frame-Options` or `X-Content-Type-Options`.

EXPECTED RESULT

The first line reads `HTTP/1.1 200 OK`, followed by headers and the HTML of the page. Content-Type is `text/html`.

WHY

`-i` tells curl to include response headers. Headers reveal technology hints and which browser protections the server requests.

Student question: Does a missing security header automatically prove that an application is exploitable? Explain your answer.

Exercise 3 — Map the application

Open Juice Shop and manually explore the interface. Locate the product catalog, search, login page, registration page, shopping cart, feedback/contact form and account pages. Record the location and purpose of each, then build an attack surface inventory:

FEATURE	INPUT OR INTERACTION	POSSIBLE SECURITY CONCERN
Login	Username and password	Authentication
Search	Search query	Input handling
Product page	Product selection	Data exposure
Shopping cart	Cart operations	Authorization
Feedback	User-entered text	Output encoding

These are **areas to investigate**, not confirmed vulnerabilities.

Exercise 4 — Examine the browser

1. Press **F12** to open Developer Tools and select the **Network** tab.
2. Reload Juice Shop.
3. Observe HTTP requests, JavaScript resources, API calls (look for paths beginning `/api/` or `/rest/`), response codes and request methods.
4. Record three requests and describe the purpose of each.

EXPECTED RESULT

Dozens of requests appear, including JSON API calls such as a product list request.

WHY

Modern apps load a JavaScript shell and then fetch data from APIs. Those API endpoints are the real attack surface — often larger than what the visible interface suggests.

Student assignment

Create a one-page reconnaissance report containing:

1. Target description and scope.
2. Open ports.
3. Observed technologies, clearly distinguishing confirmed facts from guesses.
4. Five application features.
5. Three areas that deserve further security testing.

Knowledge check

1. What is the difference between active and passive reconnaissance?
2. Why is scope authorization necessary?
3. What does an open TCP port tell us?
4. Why should a security researcher record HTTP headers?
5. Why does reconnaissance not automatically establish a vulnerability?

What you learned

You discovered services, examined web traffic and mapped an application's potential attack surface — and learned that reconnaissance findings must be validated before being described as vulnerabilities.

Real-world application

A researcher may discover an overlooked login interface, an undocumented API endpoint or a forgotten component within an authorized program's scope. These discoveries help prioritize testing, but do not automatically qualify for rewards.

LAB 02

Understanding HTTP and Intercepting Requests with Burp Suite

Duration · 2–3 hours | Difficulty · Beginner | Target · Juice Shop | Tools · Burp Suite

Learning objectives

- Understand HTTP requests and responses.
- Distinguish GET and POST methods.
- Examine request headers and bodies.
- Use Burp Suite Proxy and Repeater.
- Modify harmless requests to observe application behaviour.

Background

Browsers communicate with web applications using HTTP or HTTPS. A request contains a **method**, a **path**, **headers** and an optional **body**. A response contains a **status code**, headers and usually a body. Burp Suite lets researchers inspect — and change — this conversation.

Exercise 1 — Capture your first request

1. Open Burp Suite and navigate to **Proxy**.
2. Confirm **Intercept is off**, then click **Open browser**.
3. Visit http://localhost:3000.
4. Open **Proxy → HTTP history** and select the request for `/`.
5. Record the request method, Host header, request path, HTTP status code and response Content-Type.

Exercise 2 — Understand a request

HTTP REQUEST

```
GET / HTTP/1.1
Host: localhost:3000
Accept: text/html
```

1. What does GET mean?
2. What does the Host header identify?
3. What information appears in the response?
4. What is the difference between a 200 and a 404 response?

Exercise 3 — Intercept and replay a search request

1. In Burp's browser, search Juice Shop for the harmless term `apple`.
2. In HTTP history, find the resulting request (look for `/rest/products/search?q=apple`).
3. Right-click it and choose **Send to Repeater**.
4. In the **Repeater** tab, click **Send** to replay the original request.
5. Change the search value from `apple` to `banana` in the query parameter.
6. Click **Send** again and compare the two responses.

EXPECTED RESULT

Both requests return `200 OK` with JSON. The first lists apple products; the modified request lists banana products instead.

WHY

The server only sees the request — it cannot tell whether it came from the browser's search box or from Repeater. Anything the client sends is under the user's control, which is why servers must never trust client-supplied values. This is the foundation of Labs 03–05.

Exercise 4 — Compare HTTP methods

Using normal interactions (for example, viewing products and then logging in with a test account you registered), locate a **GET** and a **POST** request in HTTP history. Compare the method, where parameters appear (URL vs. body), body content and response status.

EXPECTED RESULT

GET requests carry parameters in the URL and have no body; the login POST carries a JSON body with email and password.

WHY

GET retrieves data; POST submits data to be processed. Sensitive values belong in the body because URLs are stored in logs and history.

Do not submit malicious payloads or alter other users' data.

Student assignment

Create a short HTTP analysis report containing one captured request, one captured response, a screenshot of Burp Repeater, the original and modified harmless search values, and a paragraph explaining the difference in responses.

Knowledge check

1. What is an HTTP proxy?
2. What is the purpose of Burp Repeater?
3. What is the difference between GET and POST?
4. Why might an application use cookies?
5. Why is the ability to change a request not itself a vulnerability?

What you learned

You learned how browsers communicate with servers, how researchers inspect and modify HTTP traffic, and the importance of comparing original and modified requests.

Real-world application

Researchers use intercepting proxies to investigate API behaviour, authentication workflows, authorization controls and input validation — for example, noticing that an application relies on a user-controlled parameter, then designing authorized tests to see whether the server validates it.

LAB 03

SQL Injection Fundamentals

Duration · 2–3 hours | Difficulty · Beginner | Target · WebGoat | Tools · Browser

Learning objectives

- Explain how applications interact with databases.
- Understand SQL injection.
- Identify unsafe query construction.
- Complete a guided SQL injection exercise.
- Explain how parameterized queries prevent injection.

Background

SQL injection occurs when untrusted input is interpreted as part of a database query instead of being treated strictly as data. In this unsafe example, user input is inserted directly into SQL syntax:

PYTHON · UNSAFE

```
query = (
    "SELECT * FROM products WHERE name = '"
    + user_input +
    "'"
)
```

A safer approach uses a parameterized query. The database driver sends the input separately as a value, never as SQL instructions:

PYTHON · PARAMETERIZED

```
cursor.execute(
    "SELECT * FROM products WHERE name = ?",
    (user_input,)
)
```

Exercise 1 — Open the WebGoat SQL injection lessons

1. Start WebGoat if it is not running: `sudo docker start webgoat`
2. Open http://localhost:8080/WebGoat and log in with your training account.
3. In the lesson menu, open the **Injection** category and select **SQL Injection (intro)**.

4. Read the introduction to SQL syntax and injection. Lesson names and numbering may vary by WebGoat release.

Exercise 2 — Understand basic SQL

SQL

```
SELECT * FROM employees
WHERE department = 'Sales';
```

Explain what **SELECT** does, what **FROM** identifies, what **WHERE** controls and why the quotation marks matter. Now examine this intentionally unsafe pattern:

SQL · UNSAFE PATTERN

```
SELECT * FROM employees
WHERE name = '<user input>';
```

Discuss what could happen if the input itself contains a quotation mark, followed by more SQL.

Exercise 3 — Complete a guided injection challenge

Within WebGoat's SQL Injection lesson:

1. Read the lesson's challenge objective.
2. Identify the input field used by the exercise.
3. Submit an ordinary value first and record the normal response.
4. Follow the lesson's guided injection example.
5. Observe how the application responds when your input changes the SQL expression.
6. Complete the lesson's success condition (the lesson marker turns green).
7. Record the result and explain the cause.

EXPECTED RESULT

An ordinary value returns one matching record or an empty result. The guided input causes the query to return rows it should not — typically every row in the table — and WebGoat confirms the lesson is solved.

WHY

Your quotation mark closes the string the developer opened, and the text after it is parsed as SQL logic. A condition that is always true (for example `'1'='1'`) makes the WHERE clause match everything. The database cannot tell the developer's SQL from yours because they arrived as one string.

Only use the supplied WebGoat challenge data. Do not test external databases or attempt to extract real credentials.

Exercise 4 — Understand prevention

PYTHON · UNSAFE

```
query = "SELECT * FROM users WHERE id = " + user_id
```

PYTHON · SAFER

```
cursor.execute(
    "SELECT * FROM users WHERE id = ?",
    (user_id,)
)
```

Discuss why input validation alone is not a substitute for parameterized queries.

Student assignment

Document: (1) the vulnerable input, (2) normal application behaviour, (3) the guided test performed, (4) the observed result, (5) why the behaviour demonstrates SQL injection, and (6) recommended remediation.

Knowledge check

1. What is SQL injection?
2. Why does string concatenation increase risk?
3. What is a parameterized query?

4. Can SQL injection affect confidentiality, integrity and availability?
5. Why must students avoid testing unauthorized databases?

What you learned

You learned how unsafe queries allow user input to change query behaviour, practised recognizing SQL injection in a vulnerable application, and saw that parameterized queries are the primary defence.

Real-world application

SQL injection can expose sensitive data, allow unauthorized database changes or disrupt availability. Researchers report reproducible evidence of the flaw and its impact without unnecessarily extracting sensitive information.

LAB 04

Cross-Site Scripting (XSS)

Duration · 2–3 hours | Difficulty · Beginner | Target · Local HTML file, WebGoat | Tools · Browser, text editor

Learning objectives

- Explain cross-site scripting.
- Distinguish reflected, stored and DOM-based XSS.
- Identify unsafe output handling.
- Demonstrate a harmless XSS proof of concept.
- Explain output encoding and other defences.

Background

Cross-site scripting occurs when attacker-controlled content is interpreted as executable script in another user's browser.

TYPE	HOW IT HAPPENS
Reflected	Input is included in the immediate response and executed by the browser.
Stored	Malicious content is saved by the application and later displayed to other users.
DOM-based	Client-side JavaScript handles untrusted data in an unsafe way.

Exercise 1 — Understand safe and unsafe output

JAVASCRIPT · UNSAFE

```
document.getElementById("result").innerHTML = userInput;
```

JAVASCRIPT · SAFE FOR PLAIN TEXT

```
document.getElementById("result").textContent = userInput;
```

`innerHTML` asks the browser to parse the value as HTML markup; `textContent` treats it as literal text.

Exercise 2 — Create a harmless local XSS demonstration

1. Open a plain text editor (VS Code, gedit, Notepad or TextEdit in plain-text mode).

2. Create a file named `xss-lab.html` and paste:

XSS-LAB.HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>XSS Training Lab</title>
</head>
<body>
  <h1>Search Result</h1>

  <input id="search" type="text">
  <button onclick="showResult()">Search</button>

  <div id="result"></div>

  <script>
    function showResult() {
      const value =
        document.getElementById("search").value;

      document.getElementById("result").innerHTML =
        value;
    }
  </script>
</body>
</html>
```

3. Save the file and open it by double-clicking it (or drag it into the browser).
4. Enter `Hello World` and click **Search**. The text appears below the box.

Exercise 3 — Demonstrate script execution

In the same page, enter the following and click **Search**:

TEST INPUT

```
<img src=x onerror=alert('XSS Lab')>
```

EXPECTED RESULT

A browser alert reading *XSS Lab* appears, and a broken-image icon shows in the result area.

WHY

`innerHTML` turned your text into a real `<img>` element. The image source `x` fails to load, which fires the `onerror` handler — attacker-supplied JavaScript. (A plain `<script>` tag would not run here, because browsers do not execute scripts inserted via innerHTML — which is why event-handler payloads are used.) This proof of concept does not steal data, contact any external service or affect other users.

Exercise 4 — Fix the vulnerability

In `xss-lab.html`, replace `.innerHTML` with `.textContent`:

FIXED LINE

```
document.getElementById("result").textContent =
    value;
```

Save, reload the page and repeat the test.

EXPECTED RESULT

No alert. The payload appears on screen as ordinary text, exactly as typed.

WHY

The angle brackets are now displayed as characters, not parsed as tags, so no element — and no event handler — is ever created. The fix addresses the root cause (unsafe output) rather than trying to block specific payloads.

Exercise 5 — Explore WebGoat XSS

In WebGoat, open **Cross Site Scripting** (under the Injection category). Follow the guided introductory exercise and compare its vulnerable behaviour with your local demonstration.

Student assignment

- A screenshot of the harmless alert.
- A screenshot showing the payload displayed safely after remediation.
- An explanation of why the original code was vulnerable.

- A comparison of reflected, stored and DOM-based XSS.
- A recommended defensive coding approach.

Knowledge check

1. What does XSS stand for?
2. What is the difference between stored and reflected XSS?
3. Why can `innerHTML` be dangerous with untrusted input?
4. Why is `textContent` safer when displaying plain text?
5. Can a Content Security Policy replace correct output handling?

What you learned

You saw that browsers can interpret untrusted content as executable code, demonstrated a harmless proof of concept, and fixed the underlying unsafe output.

Real-world application

XSS may enable unauthorized actions in a user's browser, expose information accessible to scripts, or compromise application workflows. Responsible researchers demonstrate the issue with harmless evidence and never steal sessions or interact with real user accounts.

LAB 05

Broken Access Control and Writing a Bug Bounty Report

Duration · 2–4 hours | Difficulty · Beginner | Target · Local Python server | Tools · Python 3, curl

Learning objectives

- Explain authentication versus authorization.
- Recognize insecure direct object references (IDOR).
- Test access control using two training identities.
- Record reproducible evidence.
- Write a professional vulnerability report.

Background

Authentication answers *"Who are you?"* **Authorization** answers *"What are you allowed to access?"* An IDOR vulnerability occurs when an application exposes an object identifier but fails to verify that the requesting user may access that object:

```
/api/orders/101
/api/orders/102
```

Changing an identifier should never be sufficient to access another user's private order.

Exercise 1 — Create the access-control training application

Create a folder for this lab, then a file named `idor_lab.py` inside it, and paste the code below. Python is indentation-sensitive — keep the spacing exactly as shown.

IDOR_LAB.PY

```
from http.server import (
    BaseHTTPRequestHandler,
    HTTPServer
)
from urllib.parse import urlparse, parse_qs
import json

orders = {
    "101": {"owner": "alice", "item": "Notebook"},
    "102": {"owner": "bob", "item": "Keyboard"}
}

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        parsed = urlparse(self.path)

        if parsed.path != "/order":
            self.send_error(404)
            return

        params = parse_qs(parsed.query)
        user = params.get("user", [""])[0]
        order_id = params.get("id", [""])[0]

        order = orders.get(order_id)

        if not order:
            self.send_error(404)
            return

        # Intentionally vulnerable:
        # No authorization check.
        response = json.dumps(order).encode()

        self.send_response(200)
        self.send_header(
            "Content-Type", "application/json"
        )
        self.send_header(
            "Content-Length", str(len(response))
        )
        self.end_headers()
        self.wfile.write(response)

HTTPServer(
    ("127.0.0.1", 8000), Handler
).serve_forever()
```

The deliberately simplified `user` parameter represents the requester. It is not a real authentication system.

Exercise 2 — Start the application

TERMINAL 1 — FROM THE LAB FOLDER

```
python3 idor_lab.py
```

The terminal shows no output and does not return to a prompt — the server is running. Leave it open and open a **second** terminal for the next steps.

Exercise 3 — Access an authorized training record

TERMINAL 2

```
curl "http://127.0.0.1:8000/order?user=alice&id=101"
```

EXPECTED RESULT

`{"owner": "alice", "item": "Notebook"}`

WHY

Alice is viewing the record assigned to Alice. This is the legitimate baseline — always establish normal behaviour before testing.

Exercise 4 — Demonstrate broken authorization

TERMINAL 2

```
curl "http://127.0.0.1:8000/order?user=alice&id=102"
```

EXPECTED RESULT

`{"owner": "bob", "item": "Keyboard"}` — Bob's order, returned to Alice.

WHY

The server looks the order up by ID and returns it, but never compares the order's `owner` with the requester. Knowing (or guessing) an ID is enough to read it — that is the IDOR.

Exercise 5 — Fix the authorization logic

Stop the server with **Ctrl+C**. In `idor_lab.py`, locate:

```
        if not order:
            self.send_error(404)
            return
```

Immediately below it, at the same indentation, add:

OWNERSHIP CHECK

```
        if order["owner"] != user:
            self.send_error(403, "Forbidden")
            return
```

Save, restart with `python3 idor_lab.py`, then repeat both requests using `curl -i` so the status code is visible.

EXPECTED RESULT

Alice → order 101: **HTTP 200**. Alice → order 102: **HTTP 403 Forbidden**.

WHY

The server now checks ownership on every request before returning data. Re-running the original attack to confirm it fails is called a **retest** — it proves the fix works.

PRODUCTION SECURITY NOTE

A real application must derive the user's identity from a validated server-side session or authentication token — never from a freely editable `user` parameter. Production authorization must also account for roles, permissions and the specific resource being accessed.

Exercise 6 — Write a professional bug bounty report

Use the template below. Every field should allow a developer who has never seen your work to reproduce and fix the issue.

VULNERABILITY REPORT · TEMPLATE

Broken Object-Level Authorization Allows Access to Another User's Order

Target	Local IDOR Training Application
Vulnerability type	Broken Access Control / IDOR
Severity	Medium (illustrative only; actual severity depends on impact and environment)

Description. The order endpoint does not verify that the requesting user owns the requested order. A requester can retrieve another training user's order by changing the order identifier.

Steps to reproduce.

1. Start the vulnerable local training application.
2. Request order 101 as Alice.
3. Confirm that Alice's order is returned.
4. Request order 102 while still identifying as Alice.
5. Observe that Bob's order is returned.

Expected behaviour. The application should reject unauthorized access to another user's order.

Actual behaviour. The application returns the requested record without checking ownership.

Security impact. In a production application, this flaw could expose private customer records or other resources.

Recommended remediation. Enforce server-side object-level authorization on every request using a trusted authenticated identity.

Retest result. After adding the ownership check, the unauthorized request receives HTTP 403.

Student assignment

Submit a completed report containing: (1) title, (2) description, (3) reproduction steps, (4) evidence, (5) expected versus actual behaviour, (6) security impact, (7) remediation recommendation and (8) retest results.

Knowledge check

1. What is the difference between authentication and authorization?
2. What does IDOR mean?

3. Why is changing a URL parameter not automatically a vulnerability?
4. Why must authorization checks occur on the server?
5. What makes a bug bounty report useful to a developer?

What you learned

You saw how missing authorization checks expose other users' resources, then fixed the application, verified the remediation and produced a professional security report.

Real-world application

Broken access control affects customer portals, healthcare applications, financial systems, cloud platforms and enterprise APIs. Researchers frequently test whether one account can reach another's resources — but must restrict testing to accounts and data they are authorized to use.

FINAL

Practical Assessment

Scenario

You are a junior security researcher assigned to assess an intentionally vulnerable application in a private training environment. Demonstrate the techniques learned throughout this guide.

Required deliverables

1. **Reconnaissance report** — identify the application's services and attack surface.
2. **HTTP analysis** — capture and explain a request and response.
3. **SQL injection assessment** — complete a guided SQL injection lesson and document the weakness.
4. **XSS assessment** — demonstrate and remediate the local XSS example.
5. **Access control report** — reproduce, fix and report the local IDOR vulnerability.

Grading rubric

ASSESSMENT CATEGORY	POINTS
Environment setup and safety	10
Reconnaissance and documentation	15
HTTP request analysis	15
SQL injection exercise	20
XSS exercise and remediation	20
Access control and final report	20
Total	**100**

Passing score: 80 / 100. Students must also demonstrate compliance with the laboratory scope and responsible testing practices.

APPENDIX A

Instructor Guidance

Teaching recommendations

For each lab:

1. Explain the concept in simple language.
2. Demonstrate the first exercise.
3. Allow students to perform the remaining steps independently.
4. Ask students to explain what they observed.
5. Review the security impact and remediation.
6. Evaluate the submitted evidence and report.

Do not grade solely on whether a tool produced the expected response — students should demonstrate that they understand *why* the behaviour occurred.

Troubleshooting

SYMPTOM	CHECK AND FIX
Juice Shop not reachable	Run `sudo docker ps -a` and `sudo docker logs juice-shop`. Restart with `sudo docker start juice-shop`.
WebGoat not reachable	Run `sudo docker logs webgoat`. It can take a minute to initialize — wait for the startup message, and remember the path is case-sensitive: `/WebGoat`.
Burp shows a port-in-use error	WebGoat already holds 8080. Change Burp's proxy listener to 8081 (Part B, Step 4).
Burp shows no traffic	Confirm the student is using Burp's built-in browser, the listener is running, and the HTTP history filter is not hiding the requests.
"permission denied" on docker	Use `sudo`, or confirm Docker Desktop is running on Windows/macOS.
IDOR app will not start	Verify Python 3 is installed and port 8000 is free. "Address already in use" means a previous copy is still running — close that terminal. An *IndentationError* means the spacing in the pasted code was altered.

APPENDIX B

Continuing Your Bug Bounty Education

After completing this guide, you are ready to explore:

- OWASP Top 10 web application risks
- API security testing
- Authentication and session management
- Server-side request forgery (SSRF)
- File upload security
- Business logic vulnerabilities
- Responsible disclosure and severity assessment

Recommended learning resources

OWASP Web Security Testing Guide	owasp.org/www-project-web-security-testing-guide
OWASP Juice Shop	owasp.org/www-project-juice-shop
OWASP WebGoat	owasp.org/www-project-webgoat
PortSwigger Web Security Academy	portswigger.net/web-security
HackerOne Hacktivity	hackerone.com
Bugcrowd University	bugcrowd.com/hackers/bugcrowd-university

COURSE COMPLETION

Discover → Observe → Test → Validate → Remediate → Report

By completing all five labs, you have practised the essential beginner workflow of an ethical bug bounty researcher: setting up a controlled environment, identifying potential weaknesses, verifying findings without harming real systems, and documenting results professionally.

The goal is not simply to find bugs. It is to think critically, work methodically, respect authorization boundaries, and communicate security risk responsibly.

CYBERSECURITY MASTERCLASS BY JOSHUA MOSES · CYBERTRACE FOUNDATION — END OF LAB GUIDE